

```
In [59]: import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import seaborn as sns
from sklearn import preprocessing
import matplotlib.pyplot as plt
```

```
In [60]: data=pd.read_csv('android.csv')
data
```

Out[60]:

	name	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instructions-per-cycle
0	messages	0.000	5.000	0.000	10.0	1.030	164.64	146.36	0.61
1	contacts	0.014	5.000	0.601	14.0	0.072	317.32	255.84	18.07
2	gmail	0.005	0.219	0.060	6.0	0.071	247.85	266.19	11.33
3	phone	0.001	0.717	0.000	16.0	0.012	2346.59	2664.54	79.12
4	system-monitor	0.182	11.000	0.181	4.0	1.527	60.68	48.13	0.88
...	...	...	...	...	...	...	...	...	...
69	jiosaavn	0.001	9.000	0.000	9.0	1.047	148.52	128.07	0.95
70	wpsoffice	0.594	5.000	0.456	23.0	1.337	345.78	637.87	2.45
71	espacio	0.233	3.000	0.000	59.0	0.465	463.26	131.60	0.67
72	zarchiver	0.467	7.000	0.893	20.0	1.068	143.34	505.76	1.93
73	zfont3	0.758	2.000	0.047	44.0	1.109	87.56	216.57	1.48

74 rows × 10 columns

In [61]: data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74 entries, 0 to 73
Data columns (total 17 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   name                                       74 non-null     object
1   cpu-usage                                 74 non-null     float64
2   context-switch                           74 non-null     float64
3   cpu-migration                             74 non-null     float64
4   page-faults                              74 non-null     float64
5   cycles-GHz                               74 non-null     float64
6   stalled-cycles-frontend-percent          74 non-null     float64
7   stalled-cycles-backend-percent           74 non-null     float64
8   Instructions-per-cycle                    74 non-null     float64
9   stalled-cycles-per-instruction           74 non-null     float64
10  branches                                  74 non-null     float64
11  branch-misses-percent                    74 non-null     float64
12  bus-cycle                                74 non-null     int64
13  cache-misses-percent                     74 non-null     float64
14  cache-references                         74 non-null     float64
15  ref-cycles                               74 non-null     int64
16  class                                    74 non-null     int64
dtypes: float64(13), int64(3), object(1)
memory usage: 10.0+ KB
```

In [62]: data.isnull().sum()

```
Out[62]: name                    0
cpu-usage                        0
context-switch                   0
cpu-migration                    0
page-faults                     0
cycles-GHz                      0
stalled-cycles-frontend-percent  0
stalled-cycles-backend-percent  0
Instructions-per-cycle            0
stalled-cycles-per-instruction  0
branches                         0
branch-misses-percent            0
bus-cycle                       0
cache-misses-percent             0
cache-references                 0
ref-cycles                      0
class                           0
dtype: int64
```

```
In [63]: data.fillna(data.mean(), inplace=True)
data
```

Out[63]:

	name	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instructions-per-cycle
0	messages	0.000	5.000	0.000	10.0	1.030	164.64	146.36	0.61
1	contacts	0.014	5.000	0.601	14.0	0.072	317.32	255.84	18.07
2	gmail	0.005	0.219	0.060	6.0	0.071	247.85	266.19	11.33
3	phone	0.001	0.717	0.000	16.0	0.012	2346.59	2664.54	79.12
4	system-monitor	0.182	11.000	0.181	4.0	1.527	60.68	48.13	0.88
...	...	...	...	...	...	...	...	...	...
69	jiosaavn	0.001	9.000	0.000	9.0	1.047	148.52	128.07	0.95
70	wpsoffice	0.594	5.000	0.456	23.0	1.337	345.78	637.87	2.45
71	espacio	0.233	3.000	0.000	59.0	0.465	463.26	131.60	0.67
72	zarchiver	0.467	7.000	0.893	20.0	1.068	143.34	505.76	1.93
73	zfont3	0.758	2.000	0.047	44.0	1.109	87.56	216.57	1.48

74 rows × 17 columns



```
In [64]: data.shape
```

Out[64]: (74, 17)

```
In [65]: print("Total Features : ",len(data.columns)-1)
```

Total Features : 16

```
In [66]: X = data.iloc[:,[0,1,4]]
Y = data.iloc[:,[16]]
```

```
In [67]: data = data.sample(frac=1).reset_index(drop=True)
data.head()
```

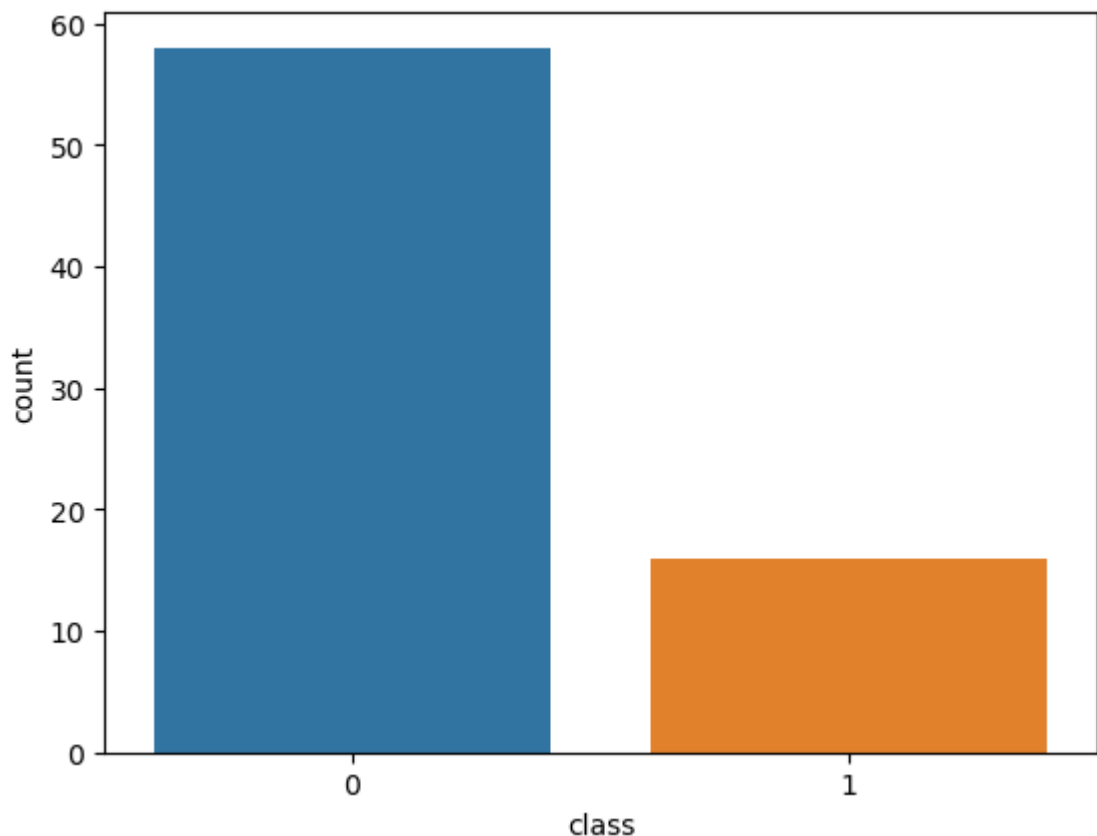
Out[67]:

	name	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instruction per-cyc
0	amazon	0.000	8.000	0.709	48.0	0.435	306.47	244.08	2.9
1	rhythmbox	0.150	2.000	0.203	5.0	1.064	54.40	41.71	0.9
2	ar services	0.142	3.000	0.291	3.0	0.956	59.38	48.31	0.6
3	discord	0.406	0.299	0.038	10.0	1.499	50.63	34.44	1.3
4	digitalwellbeing	0.015	4.000	0.453	2.0	0.046	201.94	296.21	9.0



```
In [68]: import seaborn as sns
sns.countplot(x='class',data=data)
```

Out[68]: <Axes: xlabel='class', ylabel='count'>



```
In [69]: target_count = data['class'].value_counts()
print('Bengin:', target_count[0])
print('Malware:',target_count[1])
```

Bengin: 58
Malware: 16

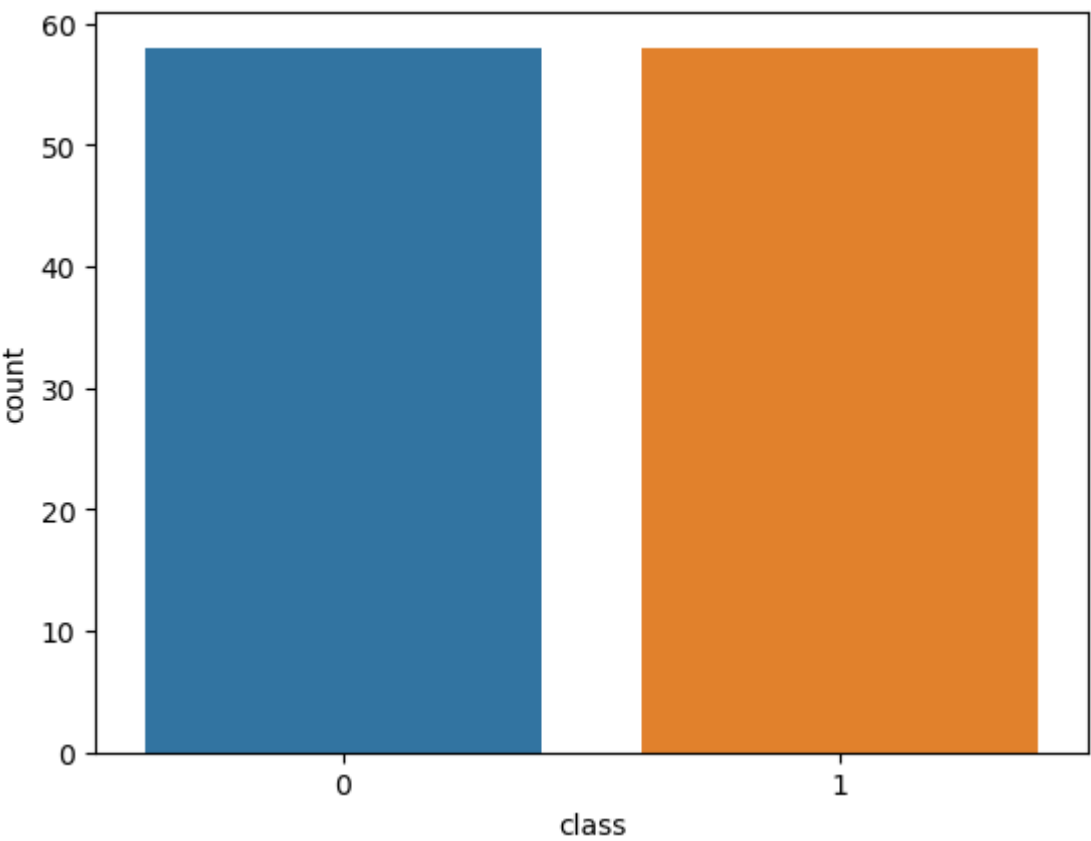
```
In [70]: count_class_0, count_class_1 = data['class'].value_counts()
df_class_0 = data[data['class'] == 0]
df_class_1 = data[data['class'] == 1]
```

```
In [71]: df_class_1_over = df_class_1.sample(count_class_0, replace=True)
df_test_over = pd.concat([df_class_0, df_class_1_over], axis=0)
df_test_over.shape
```

Out[71]: (116, 17)

```
In [72]: sns.countplot(x='class',data=df_test_over)
```

Out[72]: <Axes: xlabel='class', ylabel='count'>



```
In [73]: X=df_test_over.iloc[:,df_test_over.columns != "class"]
Y=df_test_over.iloc[:,df_test_over.columns == "class"]
X.head()
```

Out[73]:

	name	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instruction per-cyc
0	amazon	0.000	8.000	0.709	48.0	0.435	306.47	244.08	2.9
1	rhythmbox	0.150	2.000	0.203	5.0	1.064	54.40	41.71	0.9
2	ar services	0.142	3.000	0.291	3.0	0.956	59.38	48.31	0.6
3	discord	0.406	0.299	0.038	10.0	1.499	50.63	34.44	1.3
4	digitalwellbeing	0.015	4.000	0.453	2.0	0.046	201.94	296.21	9.0

```
In [74]: Y.head()
```

Out[74]:

	class
0	0
1	0
2	0
3	0
4	0

```
In [75]: from sklearn.utils import shuffle
X, Y=shuffle(X, Y)
```

```
In [76]: X.head()
```

Out[76]:

	name	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instructions-per-cycle
15	wpsoffice	0.594	5.0	0.456	23.0	1.337	345.78	637.87	2.45
33	espacio	0.233	3.0	0.000	59.0	0.465	463.26	131.60	0.67
24	flipkart	0.337	1.0	0.000	49.0	1.054	149.29	125.43	1.00
10	android accessibility site	0.107	1.0	0.099	4.0	1.211	53.00	42.81	0.93
9	gnome-software	0.883	1.0	0.124	15.0	0.003	1852.55	2496.88	85.32

```
In [77]: X=X.drop(columns='name')
X.head()
```

Out[77]:

	cpu-usage	context-switch	cpu-migration	page-faults	cycles-GHz	stalled-cycles-frontend-percent	stalled-cycles-backend-percent	Instructions-per-cycle	stalled-cycles-per-instruction
15	0.594	5.0	0.456	23.0	1.337	345.78	637.87	2.45	0.59
33	0.233	3.0	0.000	59.0	0.465	463.26	131.60	0.67	0.75
24	0.337	1.0	0.000	49.0	1.054	149.29	125.43	1.00	1.50
10	0.107	1.0	0.099	4.0	1.211	53.00	42.81	0.93	0.57
9	0.883	1.0	0.124	15.0	0.003	1852.55	2496.88	85.32	0.29

```
In [78]: Y.head()
```

Out[78]:

	class
15	1
33	1
24	1
10	0
9	0

```
In [79]: from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
bestfeatures = SelectKBest(score_func=chi2, k=10)
fit = bestfeatures.fit(X,Y)
dfscores = pd.DataFrame(fit.scores_)
dfcolumns = pd.DataFrame(X.columns)
```

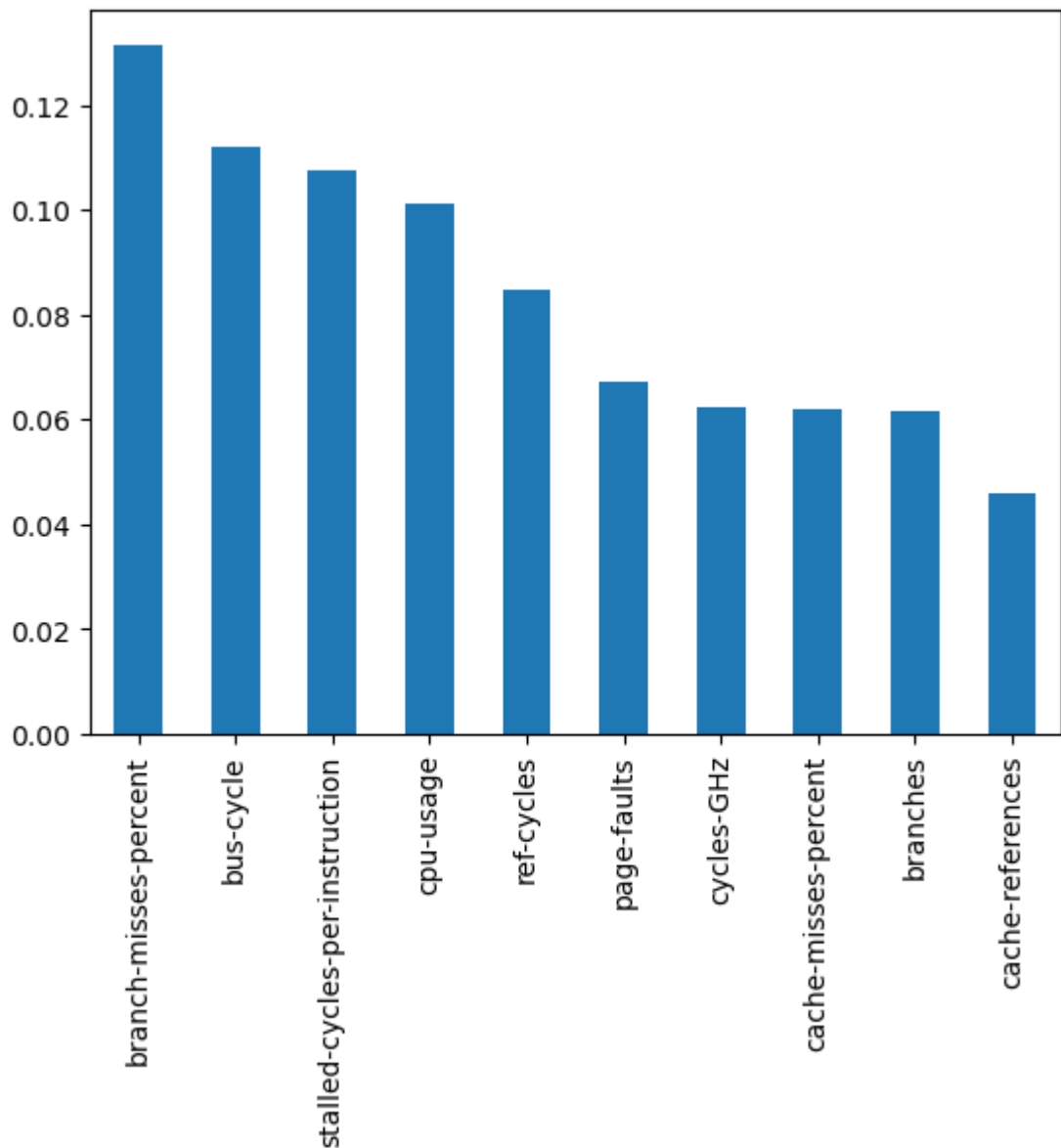
```
In [80]: featureScores = pd.concat([dfcolumns,dfscores],axis=1)
featureScores.columns = ['Specs','Score']
featureScores.nlargest(10,'Score')
```

Out[80]:

	Specs	Score
14	ref-cycles	1.286983e+06
9	branches	2.680197e+05
6	stalled-cycles-backend-percent	1.201454e+05
11	bus-cycle	1.024963e+05
5	stalled-cycles-frontend-percent	4.557351e+04
13	cache-references	8.664475e+03
7	Instructions-per-cycle	2.814869e+03
12	cache-misses-percent	2.610190e+02
3	page-faults	1.932914e+02
10	branch-misses-percent	8.292944e+01

```
In [81]: from sklearn.ensemble import ExtraTreesClassifier
import matplotlib.pyplot as plt
model = ExtraTreesClassifier()
model.fit(X,Y)
print(model.feature_importances_)
feat_importances = pd.Series(model.feature_importances_, index=X.columns)
feat_importances.nlargest(10).plot(kind='bar')
plt.show()
```

```
[0.10135676 0.03482418 0.03078419 0.06715901 0.06239553 0.02735309
 0.0400881  0.03064286 0.1073958  0.06175794 0.13139877 0.11200999
 0.06192952 0.04610066 0.08480359]
```



```
In [82]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,Y, test_size = 0.20,
```

```
In [83]: X_train.shape
```

```
Out[83]: (92, 15)
```


In [84]: `y_train.head()`

Out[84]:

	class
56	1
60	0
55	0
12	0
15	1

```
In [85]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, r

# Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42)
rf_classifier.fit(X_train, y_train)
rf_predictions = rf_classifier.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_predictions)
print(f"Random Forest Accuracy: {(rf_accuracy*100):.2f}")
```

Random Forest Accuracy: 90.00

```
In [86]: from sklearn.metrics import classification_report
rf_classification_report = classification_report(y_test, rf_predictions)
print("Classification Report for Random Forest:\n", rf_classification_repor
```

Classification Report for Random Forest:

	precision	recall	f1-score	support
0	0.94	0.88	0.91	114
1	0.85	0.93	0.89	86
accuracy			0.90	200
macro avg	0.90	0.90	0.90	200
weighted avg	0.90	0.90	0.90	200

```
In [87]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

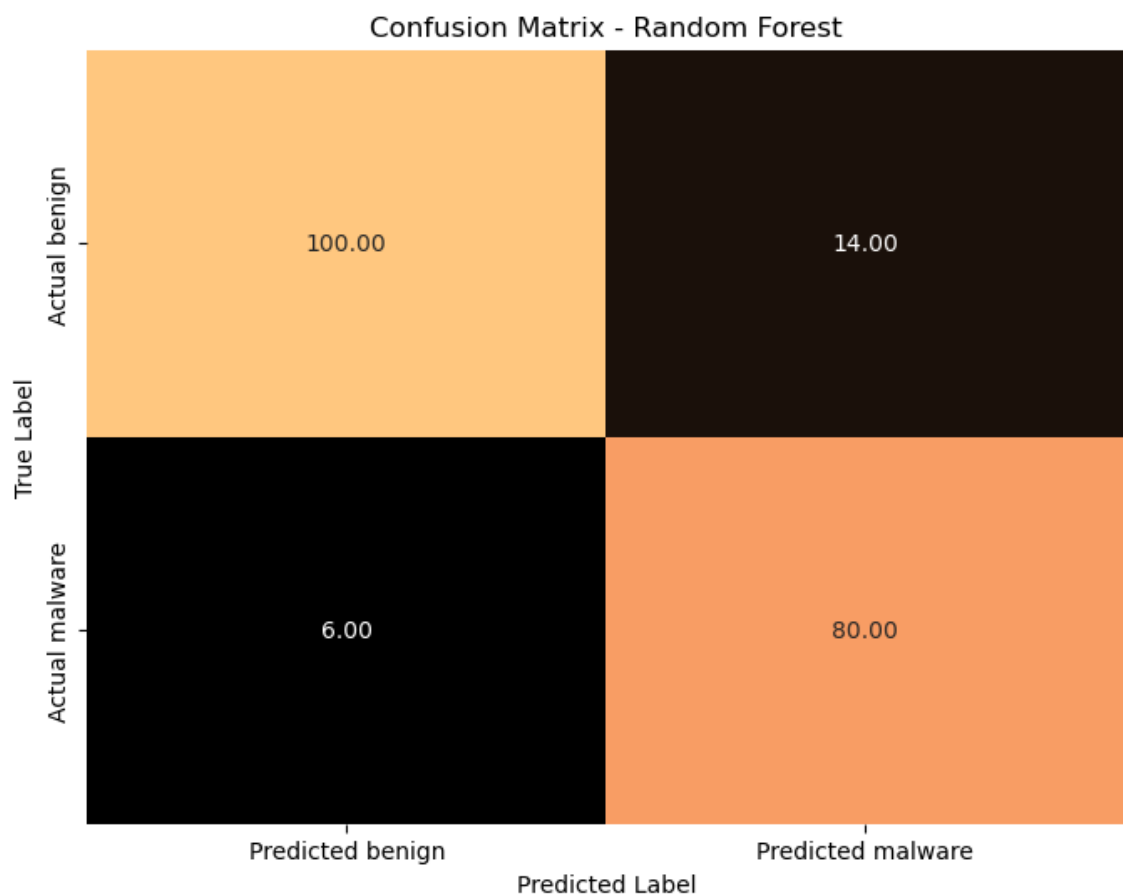
# Assuming 'benign' is represented by class 0 and 'malware' by class 1
class_labels = ['benign', 'malware']

conf_matrix = confusion_matrix(y_test, rf_predictions)
print("Confusion Matrix - Randomforest:")
print(conf_matrix)

# Plot Confusion Matrix as Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt=".2f", cmap="copper", cbar=False,
            xticklabels=[f'Predicted {label}' for label in class_labels],
            yticklabels=[f'Actual {label}' for label in class_labels])
plt.title('Confusion Matrix - Random Forest')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Confusion Matrix - Randomforest:

```
[[100  14]
 [  6  80]]
```



```
In [88]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Generate sample data
np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, r

# Support Vector Machine
svm_classifier = SVC(random_state=42)
svm_classifier.fit(X_train, y_train)
svm_predictions = svm_classifier.predict(X_test)
svm_accuracy = accuracy_score(y_test, svm_predictions)
print(f"SVM Accuracy: {svm_accuracy*100:.2f}")
```

SVM Accuracy: 94.00

```
In [89]: from sklearn.metrics import classification_report
svm_classification_report = classification_report(y_test, svm_predictions)
print("Classification Report for SVM:\n", svm_classification_report)
```

Classification Report for SVM:

	precision	recall	f1-score	support
0	0.96	0.94	0.95	114
1	0.92	0.94	0.93	86
accuracy			0.94	200
macro avg	0.94	0.94	0.94	200
weighted avg	0.94	0.94	0.94	200

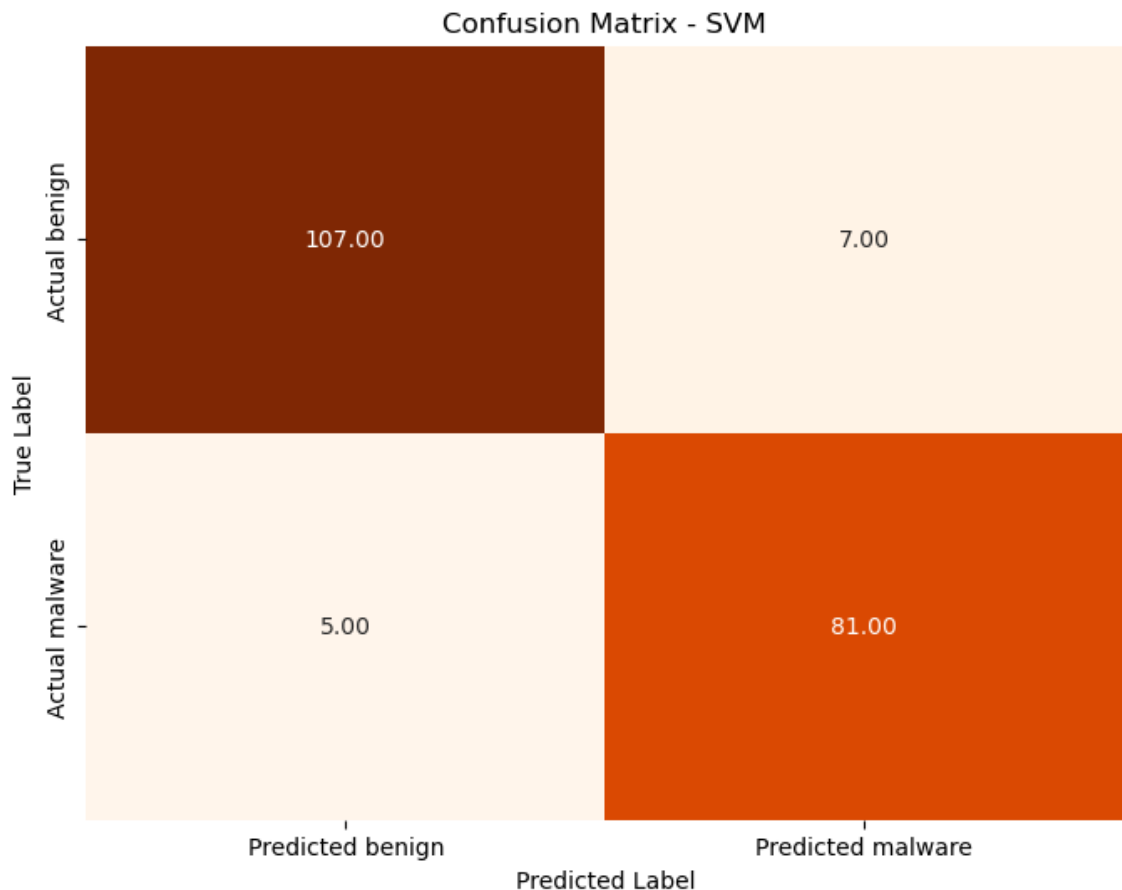
```
In [90]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming 'benign' is represented by class 0 and 'malware' by class 1
class_labels = ['benign', 'malware']
conf_matrix = confusion_matrix(y_test, svm_predictions)
print("Confusion Matrix:")
print(conf_matrix)

# Plot Confusion Matrix as Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt=".2f", cmap="Oranges", cbar=False,
            xticklabels=[f'Predicted {label}' for label in class_labels],
            yticklabels=[f'Actual {label}' for label in class_labels])
plt.title('Confusion Matrix - SVM')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Confusion Matrix:

```
[[107  7]
 [ 5 81]]
```



```
In [91]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Generate sample data
np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, r

# k-Nearest Neighbors
knn_classifier = KNeighborsClassifier(n_neighbors=11)
knn_classifier.fit(X_train, y_train)
knn_predictions = knn_classifier.predict(X_test)
knn_accuracy = accuracy_score(y_test, knn_predictions)
print(f"KNN Accuracy: {knn_accuracy*100:.2f}")
```

KNN Accuracy: 89.50

```
In [92]: from sklearn.metrics import classification_report
knn_classification_report = classification_report(y_test, knn_predictions)
print("Classification Report for k-Nearest Neighbors:\n", knn_classification_report)
```

Classification Report for k-Nearest Neighbors:

	precision	recall	f1-score	support
0	0.94	0.87	0.90	114
1	0.84	0.93	0.88	86
accuracy			0.90	200
macro avg	0.89	0.90	0.89	200
weighted avg	0.90	0.90	0.90	200

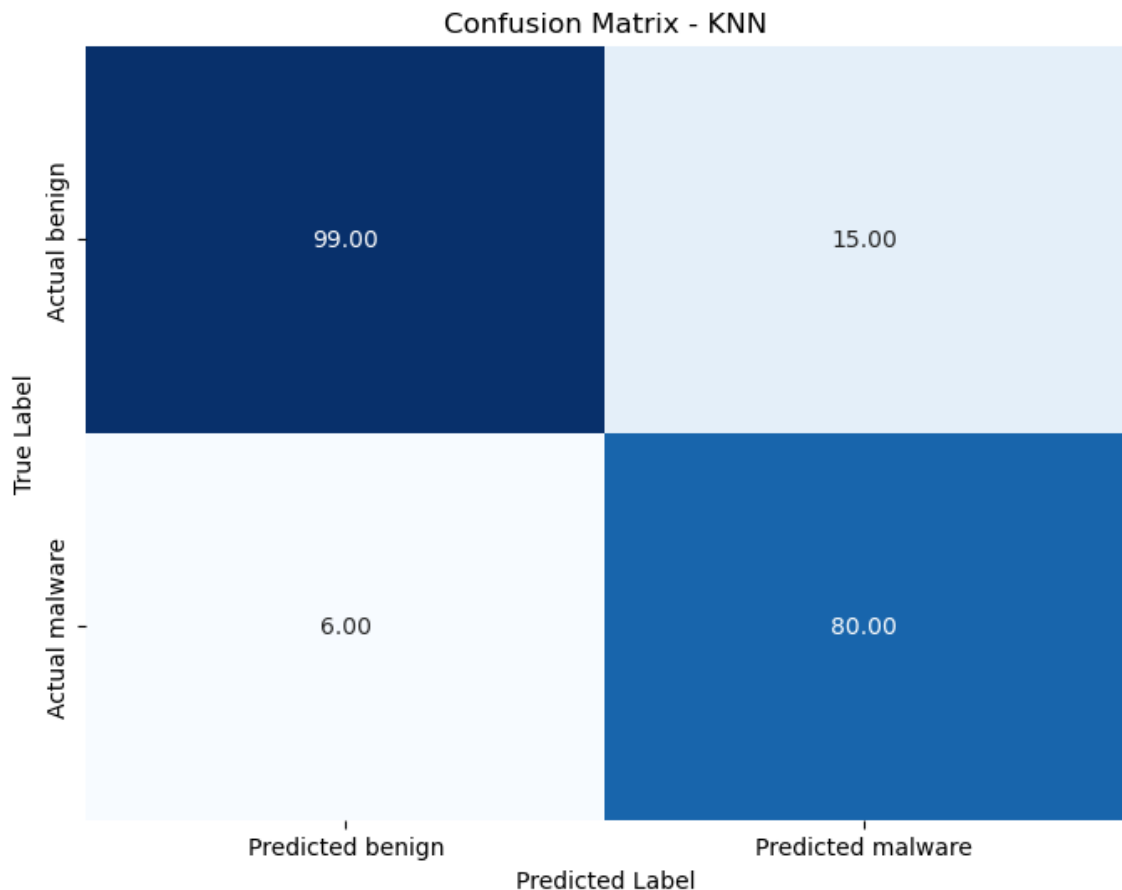
```
In [93]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming 'benign' is represented by class 0 and 'malware' by class 1
class_labels = ['benign', 'malware']
conf_matrix = confusion_matrix(y_test, knn_predictions)
print("Confusion Matrix:")
print(conf_matrix)

# Plot Confusion Matrix as Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt=".2f", cmap="Blues", cbar=False,
            xticklabels=[f'Predicted {label}' for label in class_labels],
            yticklabels=[f'Actual {label}' for label in class_labels])
plt.title('Confusion Matrix - KNN')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Confusion Matrix:

```
[[99 15]
 [ 6 80]]
```



```
In [94]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.metrics import accuracy_score

# Generate sample data
np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, r

# Define base classifiers
knn_classifier = KNeighborsClassifier(n_neighbors=11)
svm_classifier = SVC(random_state=42)
rf_classifier = RandomForestClassifier(random_state=42)

# Train base classifiers
knn_classifier.fit(X_train, y_train)
svm_classifier.fit(X_train, y_train)
rf_classifier.fit(X_train, y_train)

# Obtain predictions from base classifiers
knn_predictions = knn_classifier.predict(X_test)
svm_predictions = svm_classifier.predict(X_test)
rf_predictions = rf_classifier.predict(X_test)

# Define meta-classifier
meta_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# Stack base classifiers with the meta-classifier
stacking_classifier = StackingClassifier(
    estimators=[('knn', knn_classifier), ('svm', svm_classifier), ('rf', rf
    final_estimator=meta_classifier
)

# Train the stacking classifier
stacking_classifier.fit(X_train, y_train)

# Obtain predictions from the stacking classifier
stacking_predictions = stacking_classifier.predict(X_test)

# Evaluate accuracy
stacking_accuracy = accuracy_score(y_test, stacking_predictions)
print(f"Stacked Model Accuracy: {(stacking_accuracy * 100):.2f}")
```

Stacked Model Accuracy: 93.50


```
In [95]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Generate sample data
np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, r

# Define classifiers
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
svm_classifier = SVC(kernel='linear', random_state=42)
knn_classifier = KNeighborsClassifier(n_neighbors=11)

# Fit classifiers
rf_classifier.fit(X_train, y_train)
svm_classifier.fit(X_train, y_train)
knn_classifier.fit(X_train, y_train)

# Bagging
bagging_rf = BaggingClassifier(base_estimator=RandomForestClassifier(n_esti
bagging_svm = BaggingClassifier(base_estimator=SVC(kernel='linear', random_
bagging_knn = BaggingClassifier(base_estimator=KNeighborsClassifier(n_neigh

# Fit Bagging classifiers
bagging_rf.fit(X_train, y_train)
bagging_svm.fit(X_train, y_train)
bagging_knn.fit(X_train, y_train)

# Evaluate models
def evaluate_model(model, X_test, y_test):
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
    return accuracy

# Evaluate training accuracy
rf_train_accuracy = evaluate_model(rf_classifier, X_train, y_train)
svm_train_accuracy = evaluate_model(svm_classifier, X_train, y_train)
knn_train_accuracy = evaluate_model(knn_classifier, X_train, y_train)

bagging_rf_train_accuracy = evaluate_model(bagging_rf, X_train, y_train)
bagging_svm_train_accuracy = evaluate_model(bagging_svm, X_train, y_train)
bagging_knn_train_accuracy = evaluate_model(bagging_knn, X_train, y_train)

# Evaluate testing accuracy
bagging_rf_accuracy = evaluate_model(bagging_rf, X_test, y_test)
bagging_svm_accuracy = evaluate_model(bagging_svm, X_test, y_test)
bagging_knn_accuracy = evaluate_model(bagging_knn, X_test, y_test)

# Print results
print("Bagging Accuracies of Machine Learning Classifiers:")
print("Accuracy of RF after Bagging:", bagging_rf_accuracy * 100)
```

```
print("Accuracy of SVM after Bagging:", bagging_svm_accuracy * 100)
print("Accuracy of KNN after Bagging:", bagging_knn_accuracy * 100)
```

Bagging Accuracies of Machine Learning Classifiers:

Accuracy of RF after Bagging: 94.5

Accuracy of SVM after Bagging: 97.0

Accuracy of KNN after Bagging: 88.0

```
In [96]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier, StackingClassifier
from sklearn.metrics import accuracy_score

# Generate sample data
np.random.seed(42)
X = np.random.rand(1000, 10)
threshold = 2.5
feature_sum = np.sum(X[:, :5], axis=1)
y = (feature_sum >= threshold).astype(int)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Define base models
svm_model = SVC(kernel='linear', random_state=42)
knn_model = KNeighborsClassifier(n_neighbors=11)
rf_model = RandomForestClassifier(random_state=42)

# Define the bagging classifiers for base models
bagging_svm = BaggingClassifier(base_estimator=svm_model, random_state=42)
bagging_knn = BaggingClassifier(base_estimator=knn_model, random_state=42)
bagging_rf = BaggingClassifier(base_estimator=rf_model, random_state=42)

# Define the stacking classifier with bagging base models
estimators = [('bagging_svm', bagging_svm), ('bagging_knn', bagging_knn), ('bagging_rf', bagging_rf)]
stacking_model = StackingClassifier(estimators=estimators, final_estimator=LogisticRegression())

# Train the stacking model
stacking_model.fit(X_train, y_train)

# Make predictions on the test set
stacking_test_predictions = stacking_model.predict(X_test)

# Evaluate the performance
stacking_test_accuracy = accuracy_score(y_test, stacking_test_predictions)

print(f"Hybrid Ensemble Model Accuracy: {(stacking_test_accuracy*100):.2f}")
```

Hybrid Ensemble Model Accuracy: 96.50

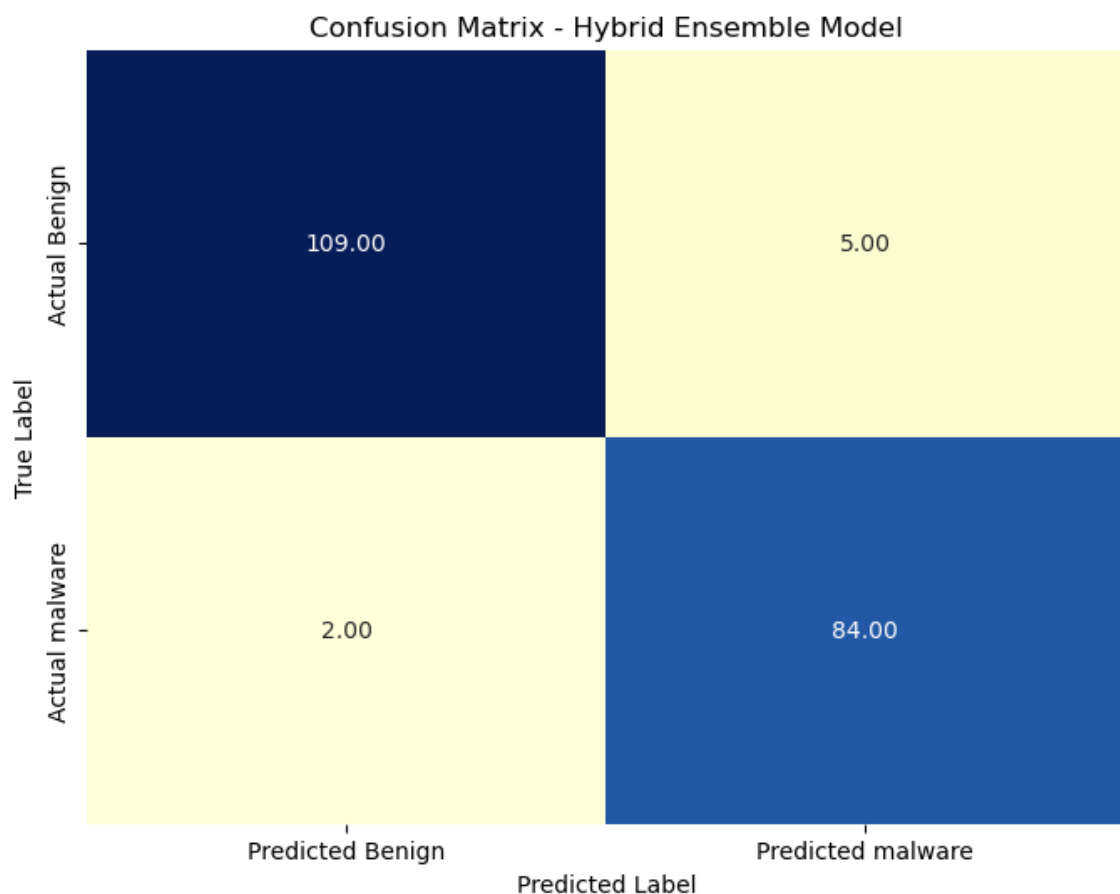
```
In [97]: from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming 'benign' is represented by class 0 and 'malware' by class 1
class_labels = ['Benign', 'malware']
conf_matrix = confusion_matrix(y_test, stacking_test_predictions)
print("Confusion Matrix:")
print(conf_matrix)

# Plot Confusion Matrix as Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt=".2f", cmap="YlGnBu", cbar=False,
            xticklabels=[f'Predicted {label}' for label in class_labels],
            yticklabels=[f'Actual {label}' for label in class_labels])
plt.title('Confusion Matrix - Hybrid Ensemble Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Confusion Matrix:

```
[[109  5]
 [  2 84]]
```



```
In [98]: # Display only 10 instances for both malicious and non-malicious
malicious_indices = np.where(stacking_test_predictions == 1)[0][:10]
non_malicious_indices = np.where(stacking_test_predictions == 0)[0][:10]

malicious_instances = X_test[malicious_indices]
non_malicious_instances = X_test[non_malicious_indices]

# Convert instances to 0 or 1 based on a threshold of 0.5
malicious_instances_binary = np.where(malicious_instances >= 0.5, 1, 0)
non_malicious_instances_binary = np.where(non_malicious_instances >= 0.5, 1, 0)

print("\nMalicious Instances:")
print(malicious_instances_binary)
print("\nNon-Malicious Instances:")
print(non_malicious_instances_binary)
```

Malicious Instances:

```
[[1 0 0 1 0 1 0 1 0 0]
 [1 1 0 1 1 0 0 1 1 0]
 [0 1 0 1 1 1 1 0 0 1]
 [1 0 0 1 1 1 1 0 1 1]
 [1 0 1 1 1 0 1 1 0 0]
 [0 0 1 1 0 0 1 0 0 1]
 [0 1 0 1 1 1 1 0 0 1]
 [1 1 1 1 1 1 1 0 0 1]
 [0 0 1 1 1 0 1 1 1 0]
 [0 1 1 0 0 1 1 1 0 0]]
```

Non-Malicious Instances:

```
[[1 1 0 1 0 0 1 0 1 0]
 [0 1 1 0 0 0 1 1 0 1]
 [1 1 0 0 1 0 1 0 0 1]
 [0 0 1 0 1 1 1 1 1 1]
 [1 1 0 0 1 0 1 0 1 1]
 [0 0 0 1 0 1 1 1 0 1]
 [0 0 0 0 0 1 0 0 1 1]
 [0 0 1 0 1 1 1 0 0 1]
 [0 0 0 0 0 1 0 1 0 1]
 [0 0 1 0 0 0 0 1 1 0]]
```

```
In [99]: import matplotlib.pyplot as plt

# Count the number of 0s and 1s in malicious instances
malicious_counts = np.bincount(malicious_instances_binary.flatten())

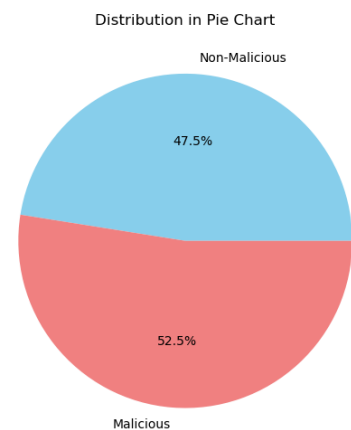
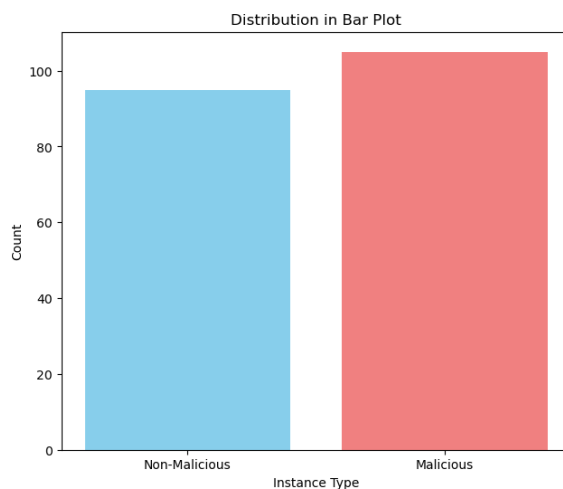
# Count the number of 0s and 1s in non-malicious instances
non_malicious_counts = np.bincount(non_malicious_instances_binary.flatten())

# Combine counts for a single pie chart and bar plot
combined_counts = malicious_counts + non_malicious_counts

# Plot side by side
plt.figure(figsize=(16, 6))

# Bar plot
plt.subplot(1, 2, 1)
plt.bar(['Non-Malicious', 'Malicious'], combined_counts, color=['skyblue',
plt.title('Distribution in Bar Plot')
plt.xlabel('Instance Type')
plt.ylabel('Count')

# Pie chart
plt.subplot(1, 2, 2)
plt.pie(combined_counts, labels=['Non-Malicious', 'Malicious'], autopct='%1
plt.title('Distribution in Pie Chart')
plt.show()
```



```

In [100]: import matplotlib.pyplot as plt
import seaborn as sns

models_group1 = ['RF', 'SVM', 'KNN']
accuracy_scores_group1 = [90.0, 94.00, 89.50]

models_group2 = ['Stacked Model', 'Bagged Stacking Model']
accuracy_scores_group2 = [93.50, 96.50]

# Set up the figure with subplots
fig, axes = plt.subplots(1, 2, figsize=(14,6))

# Group 1 Bar Plot
sns.barplot(x=models_group1, y=accuracy_scores_group1, palette="pastel", ax=axes[0])
axes[0].set_ylabel('Accuracy (%)')
axes[0].set_xlabel('Individual Models')
axes[0].set_title('Distribution of Accuracy Scores (RF, SVM, KNN)')

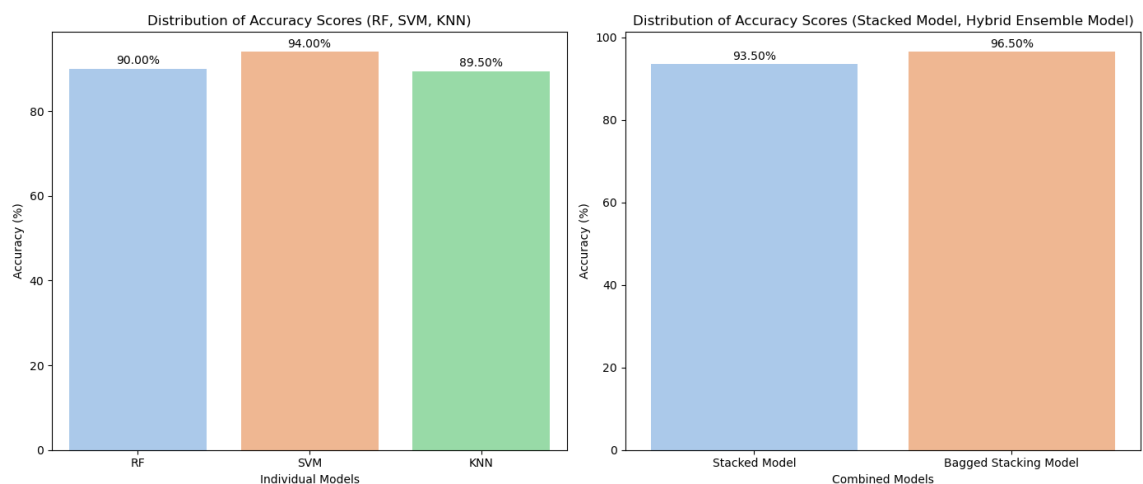
# Add accuracy values on top of the bars for Group 1
for i, v in enumerate(accuracy_scores_group1):
    axes[0].text(i, v + 0.5, f'{v:.2f}%', ha='center', va='bottom', color='black')

# Group 2 Bar Plot
sns.barplot(x=models_group2, y=accuracy_scores_group2, palette="pastel", ax=axes[1])
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_xlabel('Combined Models')
axes[1].set_title('Distribution of Accuracy Scores (Stacked Model, Hybrid Ensemble Model)')

# Add accuracy values on top of the bars for Group 2
for i, v in enumerate(accuracy_scores_group2):
    axes[1].text(i, v + 0.5, f'{v:.2f}%', ha='center', va='bottom', color='black')

plt.tight_layout()
plt.show()

```



```

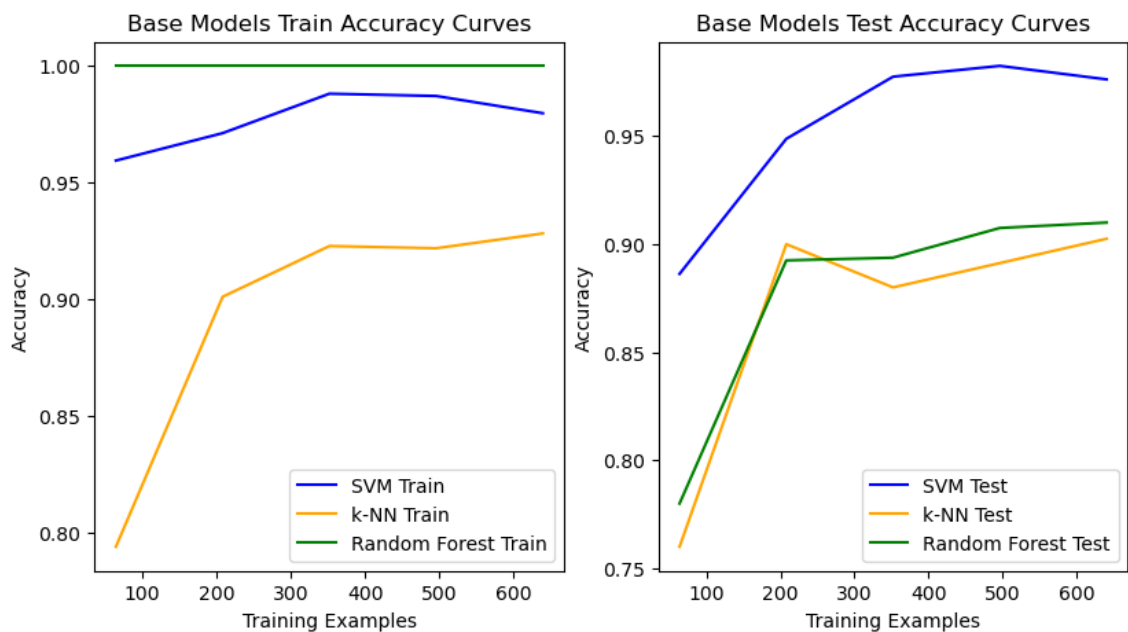
In [101]: # Learning curves for base models
plt.figure(figsize=(15, 5))

for model, name, color in zip([svm_model, knn_model, rf_model], ['SVM', 'k-
    train_sizes, train_scores = learning_curve(
        model, X_train, y_train, cv=5, n_jobs=-1, train_sizes=np.linspace(0
    )
    train_scores_mean = np.mean(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)

    plt.subplot(1, 3, 1)
    plt.plot(train_sizes, train_scores_mean, label=f'{name} Train', color=c
    plt.title('Base Models Train Accuracy Curves')
    plt.xlabel('Training Examples')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 3, 2)
    plt.plot(train_sizes, test_scores_mean, label=f'{name} Test', color=col
    plt.title('Base Models Test Accuracy Curves')
    plt.xlabel('Training Examples')
    plt.ylabel('Accuracy')
    plt.legend()

```



```
In [102]: # Learning curve for HYE model
train_sizes, train_scores, test_scores = learning_curve(
    stacking_model, X_train, y_train, cv=5, n_jobs=-1, train_sizes=np.linspace(
)
train_scores_mean = np.mean(train_scores, axis=1)
test_scores_mean = np.mean(test_scores, axis=1)

fig, ax = plt.subplots(figsize=(12, 6))
plt.plot(train_sizes, train_scores_mean, label='HYE Train', linestyle='--', c='red')
plt.plot(train_sizes, test_scores_mean, label='HYE Test', linestyle='--', c='purple')
plt.title('Hybrid Ensemble Learning Curves')
plt.xlabel('Training Examples')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```

